

QUESTION 14

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~/oss
nano q14.c
vedsk0831@DESKTOP-83FU7HB:~/oss$ gcc -Wall -Wextra q14.c -o q14
vedsk0831@DESKTOP-83FU7HB:~/oss$ ./q14
Q14 completed successfully.
stdout -> append.txt
stderr -> errors.txt
vedsk0831@DESKTOP-83FU7HB:~/oss$ cat append.txt
This line is appended to append.txt
vedsk0831@DESKTOP-83FU7HB:~/oss$ ./q14
Q14 completed successfully.
stdout -> append.txt
stderr -> errors.txt
vedsk0831@DESKTOP-83FU7HB:~/oss$ cat append.txt
This line is appended to append.txt
This line is appended to append.txt
vedsk0831@DESKTOP-83FU7HB:~/oss$ cat errors.txt
ls: cannot access '/directory_that_does_not_exist': No such file
or directory
vedsk0831@DESKTOP-83FU7HB:~/oss$ ls -l append.txt errors.txt
-rw-r--r-- 1 vedsk0831 vedsk0831 72 Sep 26 03:46 append.txt
-rw-r--r-- 1 vedsk0831 vedsk0831 78 Sep 26 03:46 errors.txt
vedsk0831@DESKTOP-83FU7HB:~/oss$ |
```

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int output_fd;
```

```
    int error_fd;
```

```
    pid_t pid;
```

```
    /* Open append.txt in append mode */
```

```
output_fd = open(
    "append.txt",
    O_WRONLY | O_CREAT | O_APPEND,
    0644
);
```

```
if (output_fd == -1)
{
    perror("Error opening append.txt");
    return 1;
}
```

```
/* Open errors.txt */
error_fd = open(
    "errors.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);
```

```
if (error_fd == -1)
{
    perror("Error opening errors.txt");
    close(output_fd);
    return 1;
}
```

```
/* Create child process */
pid = fork();
```

```
if (pid < 0)
```

```
{
```

```
    perror("fork");
```

```
    close(output_fd);
```

```
    close(error_fd);
```

```
    return 1;
```

```
}
```

```
/* Child process */
```

```
if (pid == 0)
```

```
{
```

```
    /*
```

```
     * Redirect stdout to append.txt
```

```
    */
```

```
    if (dup2(output_fd, STDOUT_FILENO) == -1)
```

```
    {
```

```
        perror("dup2 stdout");
```

```
        exit(1);
```

```
    }
```

```
/*
```

```
 * Redirect stderr to errors.txt
```

```
*/
```

```
if (dup2(error_fd, STDERR_FILENO) == -1)
```

```
{
```

```
    perror("dup2 stderr");
```

```
    exit(1);
```

```
}
```

```
/* Close original descriptors */
```

```
close(output_fd);
```

```
close(error_fd);
```

```
/*
```

```
 * Write directly to stdout.
```

```
 * dprintf avoids buffering problems.
```

```
*/
```

```
dprintf(
```

```
    STDOUT_FILENO,
```

```
    "This line is appended to append.txt\n"
```

```
);
```

```
/*
```

```
 * Intentionally execute ls on a
```

```
 * non-existent directory.
```

```
 * Its error goes to stderr.
```

```
*/
```

```
execlp(
```

```
    "ls",
```

```
    "ls",
```

```
    "/directory_that_does_not_exist",
```

```
    NULL
```

```
);
```

```
/*
```

```
    * Runs only if execlp() fails.
    */
    perror("execlp");
    exit(1);
}

/* Parent process */

close(output_fd);
close(error_fd);

/* Wait for child */
waitpid(pid, NULL, 0);

printf("Q14 completed successfully.\n");
printf("stdout -> append.txt\n");
printf("stderr -> errors.txt\n");

return 0;
}
```